produces:

```
1
3
5
7
9
```

The each iterator has a special place in Ruby; we'll describe how it's used as the basis of the language's 'for' loop on page 159, and we'll see on page 110 how all the other enumerable methods are defined in terms of each. Just defining an each method can add a whole lot more functionality to your classes.

A block returns a value to the method that yields to it. The value of the last expression evaluated in the block is passed back to the method as the value of the yield expression. This is how the find method used by class Array works. (The find method is actually defined in module Enumerable, which is mixed into class Array.) Its implementation would look something like the following:

```
class Array
  def find
    each do |value|
      return value if yield(value)
    end
    nil
  end
end

[1, 3, 5, 7, 9].find { |number| number * number > 30 } # => 7
```

Let's break this down. The find method is defined here as an instance method of Array. The last line of code creates a literal array, [1, 3, 5, 7, 9], and sends it the find method.

The find method uses each to pass successive elements of the array to the associated block. You can assume that the each method here is being called as if on the same array instance, as if it was [1, 3, 5, 7, 9].each, we'll talk about why that is so in Method Receiver, on page 87.

So, each takes its own block, and passes each value to that block. On the first iteration, that'll be the first element of the array, or 1. We then get to the line return value if yield(value). The first part evaluated is yield(value), which passes control to the block argument to find, namely { |number| number * number > 30 }. With 1 as the argument that's 1 * 1 > 30, which is false. Because the if clause value is false, the return value part of that line is not evaluated, and the each method goes on to the next value in the array, which in this case is 3.

If the block returns true (that is, a value other than nil or false), the method exits, returning the corresponding element, that's the return value part. In this case, that return will happen when the block gets to the element 7. If the method goes through the entire array and no element matches, the method goes to the expression after the each and returns nil—methods and blocks both return the value of their last expression.

There's a not-so-obvious piece of control flow here: when you return from inside a block, the return also acts as a return on the associated method. So when the return value finally does execute inside the each block, that value is also returned from the entire find method.